Machine Learning and Pattern Recognition

Non-Parametric Methods

Dr. Kewei "Isaac" Li

Department of Digital Engineering
Long Island University

April 18, 2025

Outline

- Projects
- Introduction
- Nearest Neighbor Method
- Tree-Based Methods

Projects

- Please self-enroll in a team on Brightspace to complete the final project.
- If you are working on your final project alone, then you still need to enroll in a team of one member (yourself).
- Final project proposal due on April 24.
- Project 3 will be posted online today and due on April 17.

Outline

- Projects
- Introduction
- Nearest Neighbor Method
- Tree-Based Methods

Introduction

We have learned **parametric models**

- Parametric models have fixed number of parameters. Once parameters are determined, it is easy to compute the output for a new input
- The parameters are determined by
  - First, assume the form of function that can map the input and output
  - Selecting a model based on the assumption
  - Estimate the parameters with training dataset
- Examples: linear regression, logistic regression, neural networks, etc.

Introduction

- Parametric models are interpretable and fast. It has some disadvantages.
- Non-parametric models do not have a fixed number of parameters.
- When constructing a non-parametric model, it never assumes the form of the function to be estimated.

Introduction

Non-Parametric models

- Non-parametric models can learn any form of the function to map input and output
- Example: nearest neighbor models, decision tree models, etc
- It has some advantages such as being more accurate in prediction. It can form any complex functions.
- The main disadvantage of the non-parametric models require a larger number of data and significantly higher computational costs

Outline

- Projects
- Introduction
- Nearest Neighbor Method
- Tree-Based Methods

Nearest Neighbor Method

- The k-nearest neighbors (KNN) algorithm is a non-parametric, supervised learning method
- While it can be used for either regression or classification, it is typically used for classification (similar points can be found near one another)
- It uses proximity to make classifications or predictions
- With KNN models, we do not process the data at training time. All the work is done at prediction time
- It is one of the popular and simplest classification and regression classifiers used in machine learning today

K-Nearest Neighbors Method

- For a set of points in space, we want to find the closest k neighbors

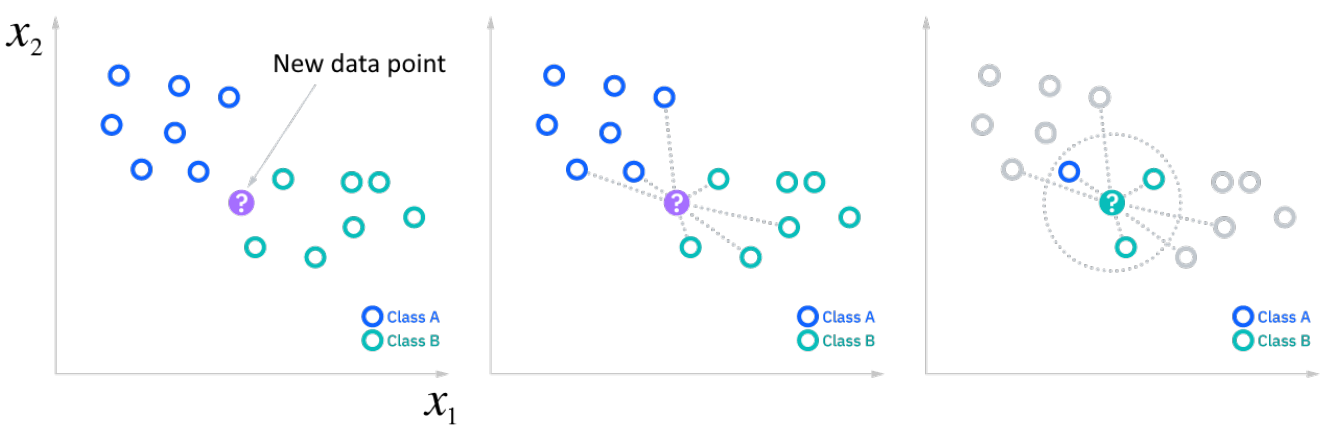


Image: www.ibm.com/topics/knn

K-Nearest Neighbors Method

- The goal of the KNN algorithm is to identify the nearest neighbors of a new data point, so that we can assign a class label to that point.
- KNN has a few requirements:
  - Determine the distance metrics
  - Choose a value for k
- Before a classification can be made, the distance must be defined. Euclidean distance is most commonly used.

K-Nearest Neighbors Method

Euclidean distance

- This is the most commonly used distance measure
- It is limited to real-valued vectors.
- It measures the distance of a straight line between the query point $\mathbf{x}$ and another point $\mathbf{y}$ in the n-dimensional space

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

K-Nearest Neighbors Method

Manhattan distance

- This is also another popular distance metric, which measures the absolute value between two points.
- It is also referred to as taxicab distance or city block distance as it is commonly visualized with a grid, illustrating how one might navigate from one address to another via city streets

$$d(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^{n} |x_i - y_i|$$

K-Nearest Neighbors Method

Minkowski distance

- This distance measure is the generalized form of Euclidean and Manhattan distance metrics.
- The parameter, p, in the formula below, allows for the creation of other distance metrics.
- Euclidean distance is represented by this formula when p is equal to 2, and Manhattan distance is denoted with p equal to 1.

$$d(\mathbf{x}, \mathbf{y}) = \left(\sum_{i=1}^{n} |x_i - y_i|^p \right)^{1/p}$$

K-Nearest Neighbors Method

Hamming distance

- This technique is used typically with Boolean or string vectors, identifying the points where the vectors do not match.
- As a result, it has also been referred to as the overlap metric.
- The Hamming distance between two equal-length strings $(\mathbf{x}, \mathbf{y})$ is the number of positions at which the corresponding symbols are different

$$H(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^{n} \delta(x_i, y_i)$$

where $\delta(a, b)$ is a function that returns 1 if $(a \neq b)$ and 0 if $(a = b)$

K-Nearest Neighbors Method

Hamming distance

- Hamming distance between two equal-length strings:
  - "kathrin" and "kerstin" is 4.
  - 0000 and 1111 is 4.
  - 2173896 and 2233796 is 3.
- Hamming distance between two integers
  - Input: $n_1 = 9, n_2 = 14$
  - $9 = 1001, 14 = 1110$
  - No. of different bits = 3 (Hamming distance)

K-Nearest Neighbors Method

The k value

- The k value defines how many neighbors will be checked to determine the prediction of a new data point.
- If $k = 1$, it will be assigned to the same class as its nearest neighbor
- Lower values of k can have high variance, but low bias, and larger values of k may lead to high bias and lower variance
- Training data with more outliers (noise) will likely perform better with higher values of k
- Overall, it is recommended to have an odd number for k to avoid ties, and cross-validation can be used to choose the optimal k value.

K-Nearest Neighbors Method

Advantages of the KNN algorithm

- Easy to implement: because of the the algorithm's simplicity and accuracy
- Adapts easily: As new training samples are added, the algorithm adjusts to account for any new data since all training data is stored into memory.
- Few hyperparameters: KNN only requires a k value and a distance metric, which is low when compared to other learning algorithms.

K-Nearest Neighbors Method

Disadvantages of the KNN algorithm

- Does not scale well: it takes up more memory and data storage compared to other classifiers.
- Curse of dimensionality: KNN doesn't perform well with high-dimensional data inputs.
- Prone to over-fitting: While feature selection and dimensionality reduction techniques are leveraged to prevent this from occurring, the value of k can also impact the model's behavior.

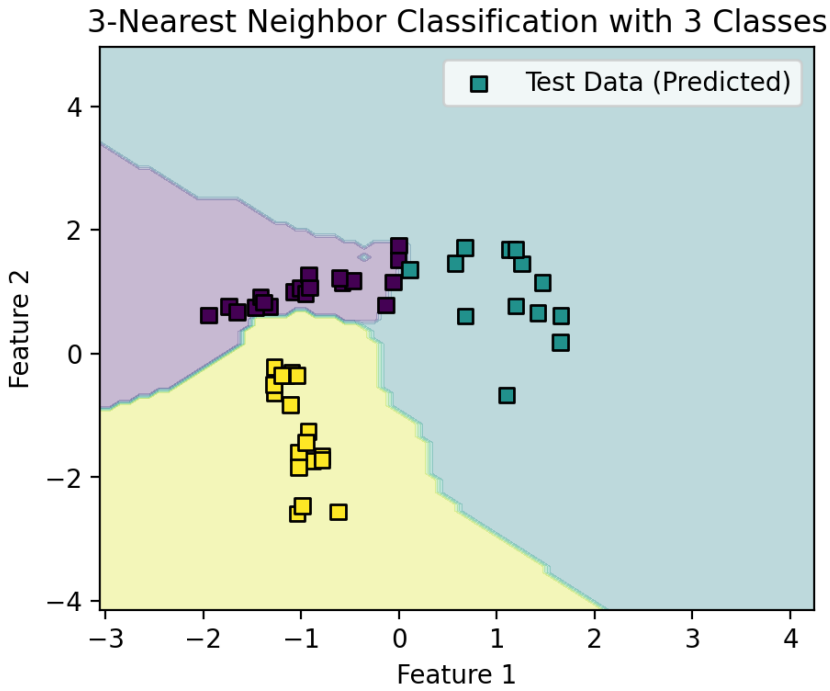
K-Nearest Neighbors Method

Applications of KNN in machine learning

- Data preprocessing: KNN algorithm can estimate for missing values in a dataset
- Recommendation Engines: Using clickstream data, KNN can provide recommendations to users on additional content.
- Finance: One study shows KNN can help banks assess risk and determine the credit-worthiness of a loan applicant
- Healthcare: to make predictions on the risk of heart attacks. The algorithm works by calculating the most likely gene expressions.
- Pattern Recognition: KNN has also assisted in identifying patterns, such as in text and digit classification.

K-Nearest Neighbors Method

- Python example for k-nearest neighbors method with $k = 3$



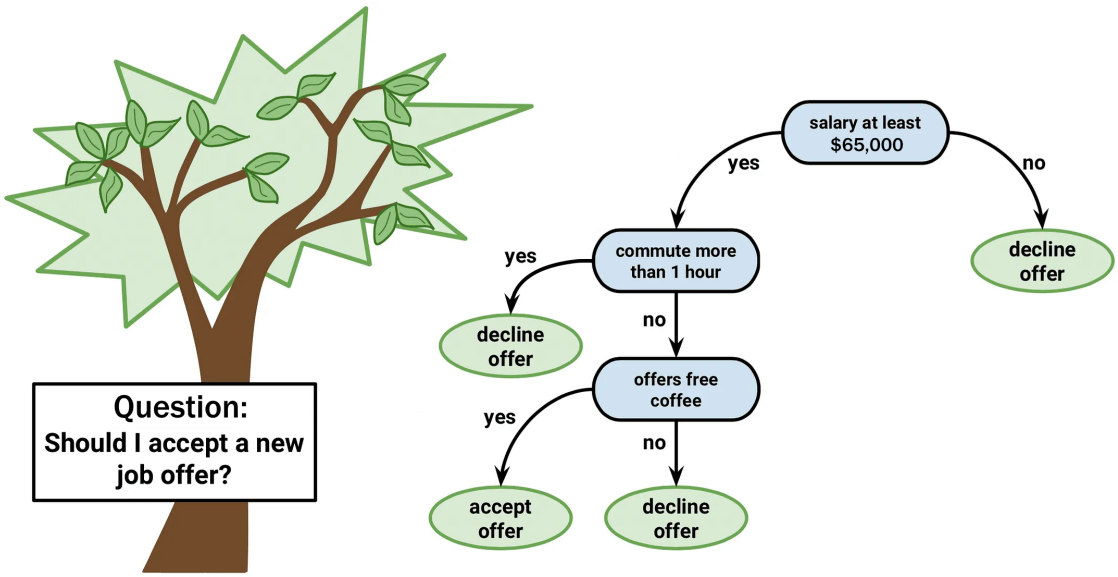
Outline

- Projects
- Introduction
- Nearest Neighbor Method
- Tree-Based Methods

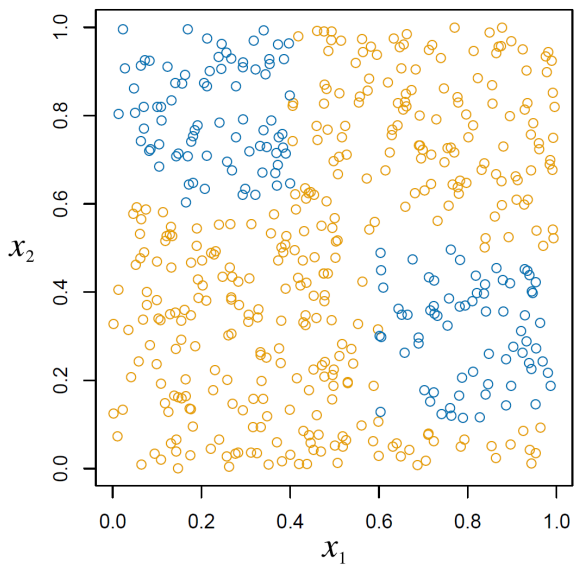
Part II:
Tree-Based Methods

Tree-Based Methods

- Tree-Based Method are powerful and versatile flowchart-like models that can be used for both prediction and classification.
- One advantage of tree-based methods: easy to understand.



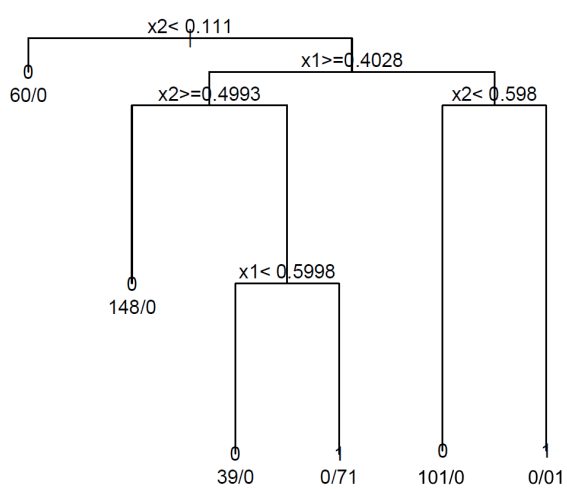
Tree-Based Methods

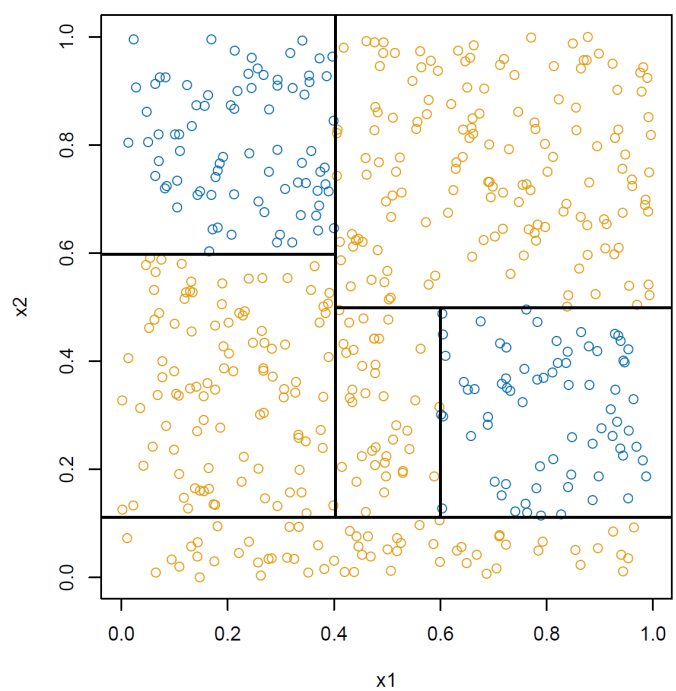


- Tree-based based methods operate by dividing up the feature space (x_1, x_2) into rectangles
- Each rectangle is like a neighborhood in the nearest-neighbor method
- In order to make a prediction for a new observation, we use the most common class in each rectangle
- Can you divide the feature space into rectangles?

Tree-Based Methods

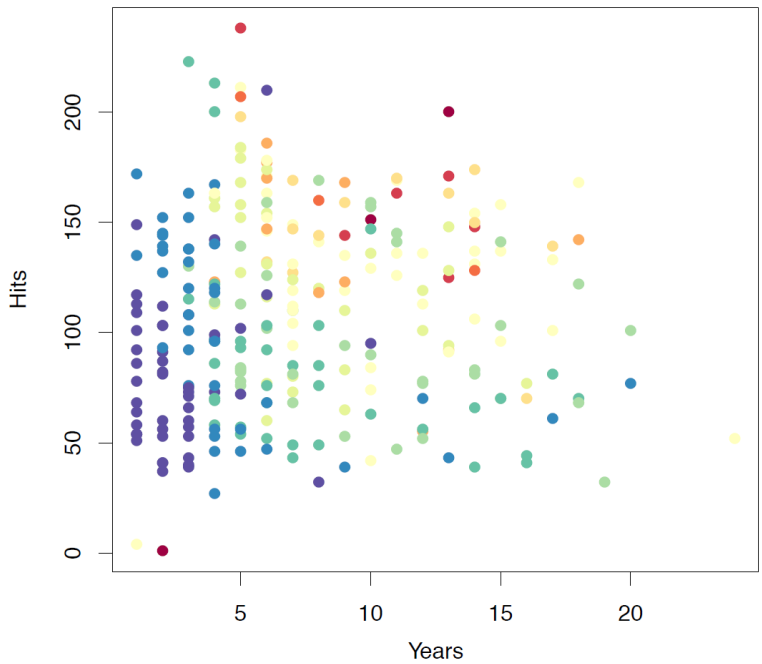
- Feature space divided into rectangles via a greedy algorithm: recursive binary splitting





Tree-Based Methods for Regression

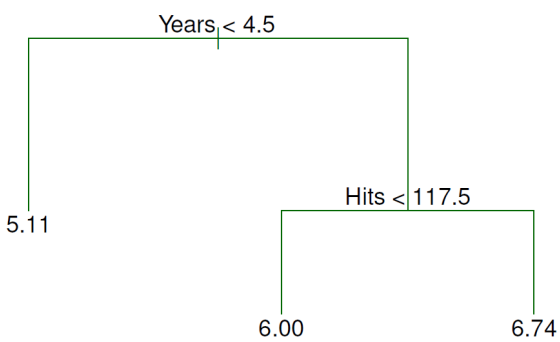
Example: predicting baseball players' salaries.

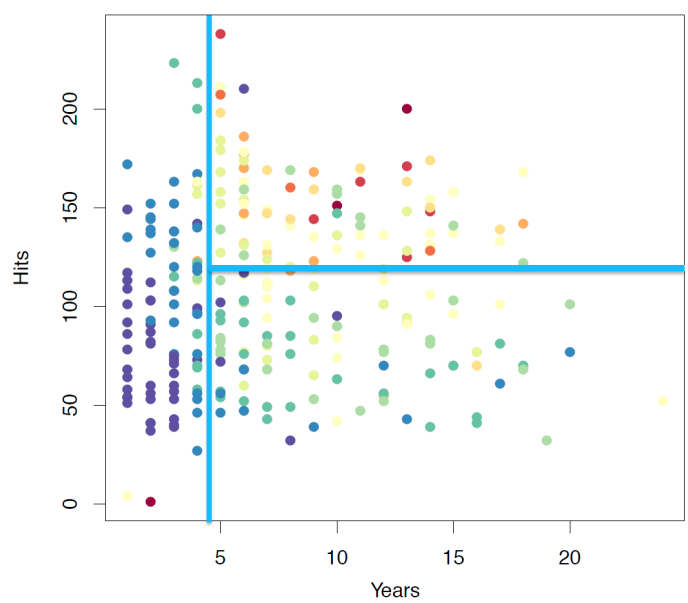


- Years (x_1): the number of years one played in the major leagues
- Hits (x_2): the number of hits one made in the previous year
- Low salary: blue, Green
- High salary: orange, red

Tree-Based Methods for Regression

- Within each of the 3 rectangles, we predict salary using the average of salary in the training data





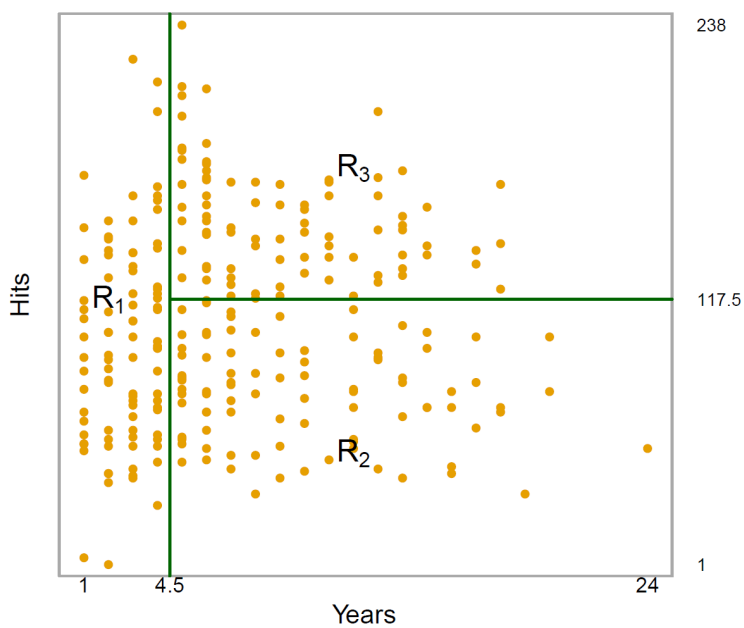
Tree-Based Methods for Regression

Example: predicting baseball players' salaries

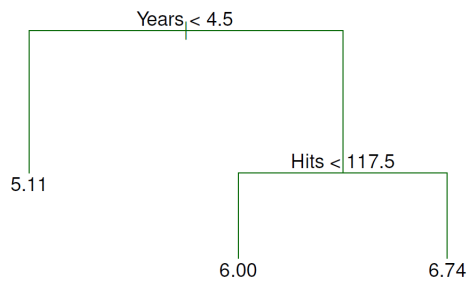
- The tree segments the players into three regions:
  - Players who have played for 4.5 or fewer years
  - Players who have played for 4.5 or more years and who made fewer than 117.5 hits last year
  - Players who have played for 4.5 or more years and who made at least 117.5 hits last year
- These three regions can be written as:
  - $R_1 = \{\mathbf{x} | \text{Years} < 4.5\}$
  - $R_2 = \{\mathbf{x} | \text{Years} > 4.5, \text{Hits} < 117.5\}$
  - $R_2 = \{\mathbf{x} | \text{Years} > 4.5, \text{Hits} > 117.5\}$

Tree-Based Methods for Regression

- Here is a diagram showing the regions as a function of Years and Hits.

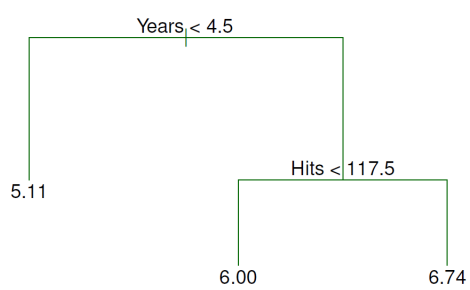


Tree-Based Methods for Regression



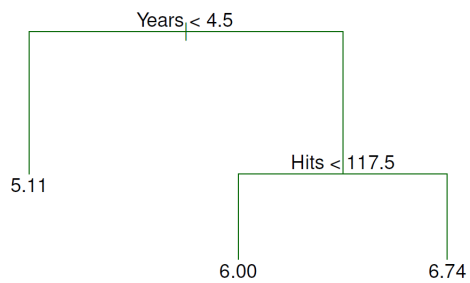
- The regions R_1, R_2, and R_3 are known as terminal nodes or leaves of the tree
- Decision trees are typically drawn upside down, in the sense that the nodes are at the bottom of the tree.
- The points along the tree where the predictor space is split are referred to as internal nodes.
- In the baseball players' salaries example, the two internal nodes are indicated by the Years < 4.5 and Hits < 117.5.

Tree-Based Methods for Regression



- We refer to the segments of the trees that connect the nodes as branches.
- The Years is the most important factor in determining Salary
- Players with less experience earn lower salaries than more experienced players.
- For less experienced players, the number of hits one made in the previous year plays little role in his salary.

Tree-Based Methods for Regression



- But among more experienced players (Years > 4.5), the number of hits one made in the previous year does affect salary
- Players who made less hits last year tend to have lower salaries.
- Players who made more hits last year tend to have higher salaries.

Tree-Based Methods for Regression

To build a regression tree, roughly speaking, there are two steps.

- We divide the feature space – that is, the set of possible values for x_1, $x_2, \ldots, x_p$ – into J non-overlapping regions, $R_1, R_2, \ldots, R_J$.
- For every new input that falls into region R_j, we make the same prediction as the mean response of the training data in R_j.

Tree-Based Methods for Regression

Example:

- Suppose we start with one region (all the baseball players in a team)
- Then, after step 1, we obtain two regions: R_1 and R_2.
- The mean response of the training data in the first region is 5, for the second region it is 6.
- Then, for a new input $\mathbf{x}$, if $\mathbf{x} \in R_1$, we will predict a value of 5, and if $\mathbf{x} \in R_2$, we will predict a value of 6.

Tree-Based Methods for Regression

How do we construct the regions $R_1, \ldots, R_J$?

- In theory, the regions could have any shape.
- However, we choose to divide the predictor space into rectangles, or boxes, for simplicity.
- The goal is to find regions $R_1, \ldots, R_J$ that minimize the Residual Sum of Squares (RSS)

$$\text{RSS} = \sum_{j=1}^{J} \sum_{i \in R_j} \left(y_i - \hat{y}_{R_j} \right)^2$$

where $\hat{y}_{R_j}$ is the mean response of the training data in the region R_j

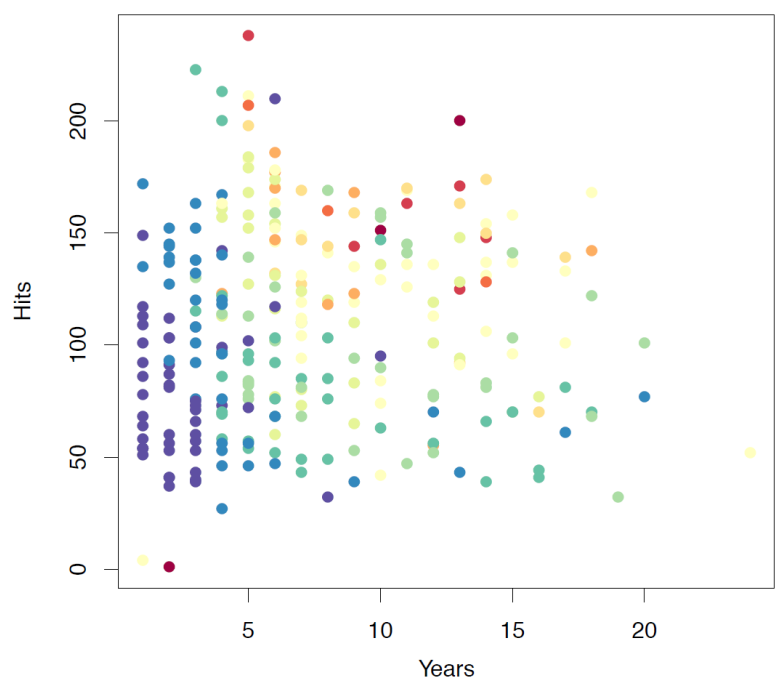
Tree-Based Methods for Regression

How do we construct the regions $R_1, \ldots, R_J$?

- Unfortunately, it is infeasible to consider every possible partition of the feature space into J boxes.
- Thus, we take a top-down, greedy approach that is known as recursive binary splitting.
- It is top-down because it begins at the top of the tree (at which we have a single region) and then successively splits the feature space
- Each split is indicated via two new branches further down on the tree.
- It is greedy because we pick the best split for the current step, rather than picking a split that will lead to a better tree in future steps

Tree-Based Methods for Regression

- In recursive binary splitting, we consider all features $x_1, x_2, \ldots, x_p$, and all possible values of the split point s for each feature



Tree-Based Methods for Regression

- Among all possible splits of every feature, we choose the feature and the split point such that the RSS is the lowest
- In general, for any j and s, we split the region by $x_j = s$ as

$$R_1(j,s) = \{\mathbf{x} | x_j < s\}, \quad R_2(j,s) = \{\mathbf{x} | x_j \geq s\}$$

and we seek the values of j and s that minimize the RSS

$$\sum_{i:x_i \in R_1} (y_i - \hat{y}_{R_1})^2 + \sum_{i:x_i \in R_2} (y_i - \hat{y}_{R_2})^2$$

Tree-Based Methods for Regression

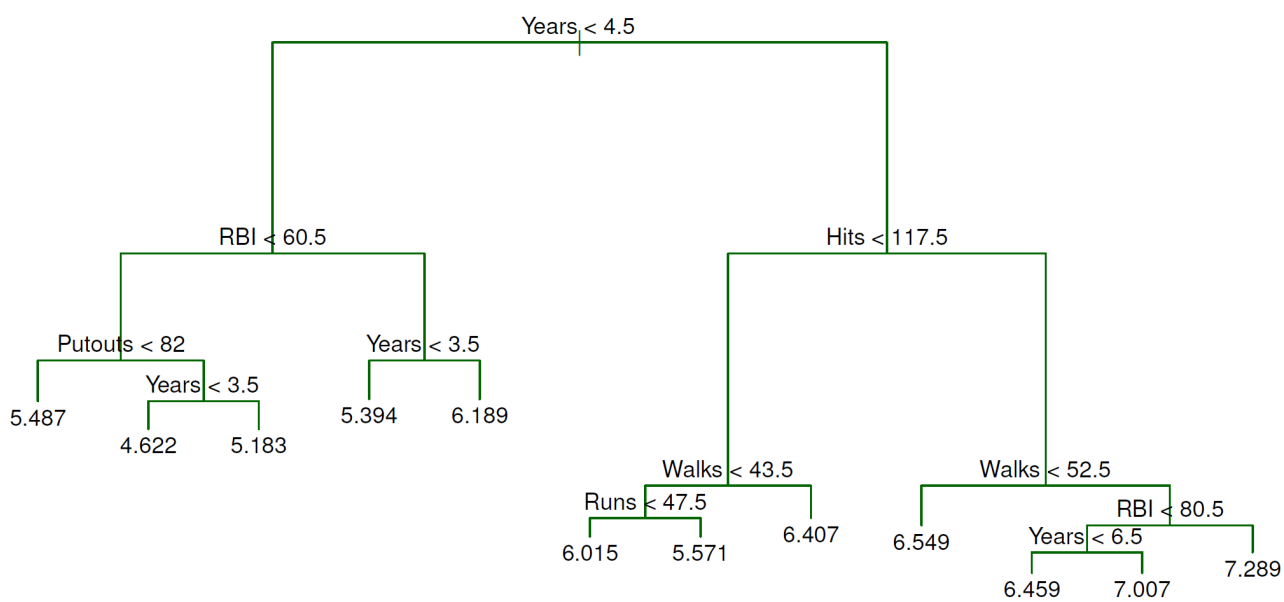
- Next, we repeat the process, looking for the best feature and best split point in order to minimize the RSS within each of the resulting regions
- After the second split, we will have three regions
- Again, we look to split one of the three regions further, so as to minimize the RSS
- The process continues until a stopping criterion is reached.
- For instance, we may continue until no region contains more than five observations

Python Code for Regression Tree

```python
def find_best_split(X, y):
    best_var = float('inf')    # positive infinity
    best_split = None
    num_features = range(X.shape[1])
    for feature_index in num_features:    # loop through each feature
        unique_features = np.unique(X[:, feature_index])
        for split_value in unique_features:   # loop each data point
            left_mask = X[:, feature_index] <= split_value
            right_mask = ~left_mask
            if np.sum(left_mask) == 0 or np.sum(right_mask) == 0:
                continue
            left_var = rss(y[left_mask])      # Compute RSS
            right_var = rss(y[right_mask])
            total_var = left_var + right_var
            if total_var < best_var:          # Save the best split
                best_var = total_var
                best_split = (feature_index, split_value)
    return best_split
```

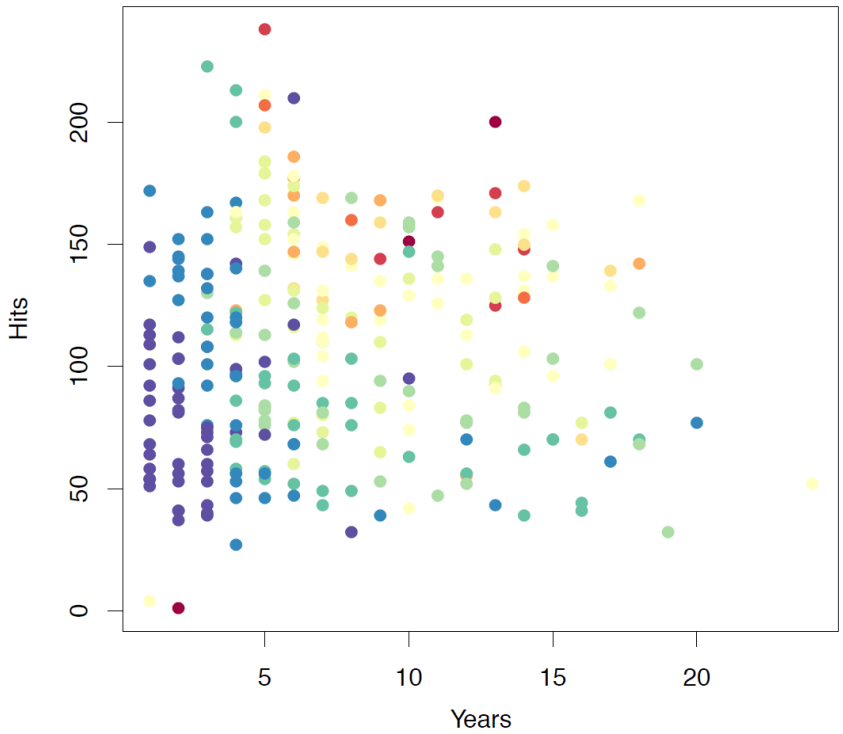

Tree-Based Methods for Regression

We could build a large tree for the baseball player example



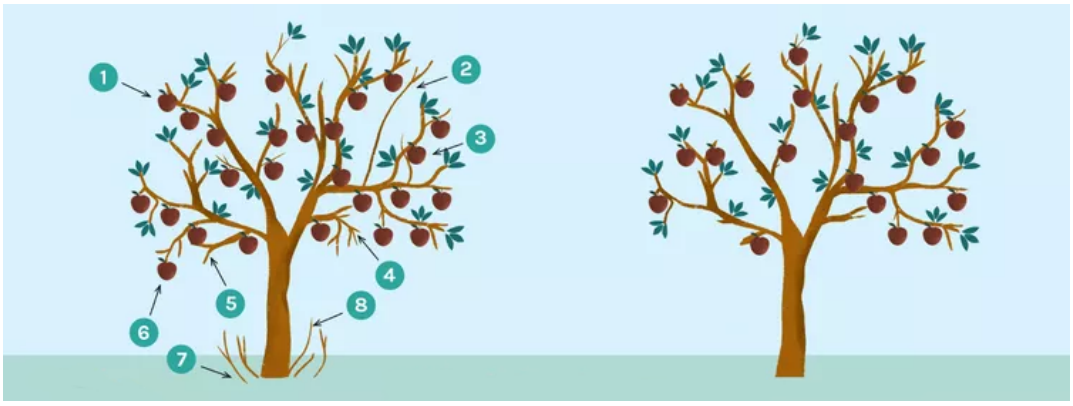
Tree-Based Methods for Regression

- If we keep going, we could over-fit the training data, and get poor test performance



Tree-Based Methods for Regression

- We could stop as soon as we cannot find a split to reduce RSS by at least some pre-specified amount
- But this strategy is short-sighted: a seemingly worthless split early on might be followed by a really good split later
- Solution: Grow a very large tree T_0, and then prune it back



Tree-Based Methods for Regression

- We will use cost complexity pruning (weakest link pruning). For a fixed α, we can find a subtree $T \subset T_0$ that (approximately) minimizes

$$C_\alpha(T) = \sum_{m=1}^{|T|} \sum_{x_i \in R_m} \left(y_i - \hat{y}_{R_m}\right)^2 + \alpha|T|$$

  where $|T|$ is the number of terminal nodes in tree T.
- The minimization process:
  - Create a sequence of subtrees by successively removing the bottom-level split that minimizes the increase in overall error, until the root is reached.
  - Return the T in the sequence that minimizes the cost complexity

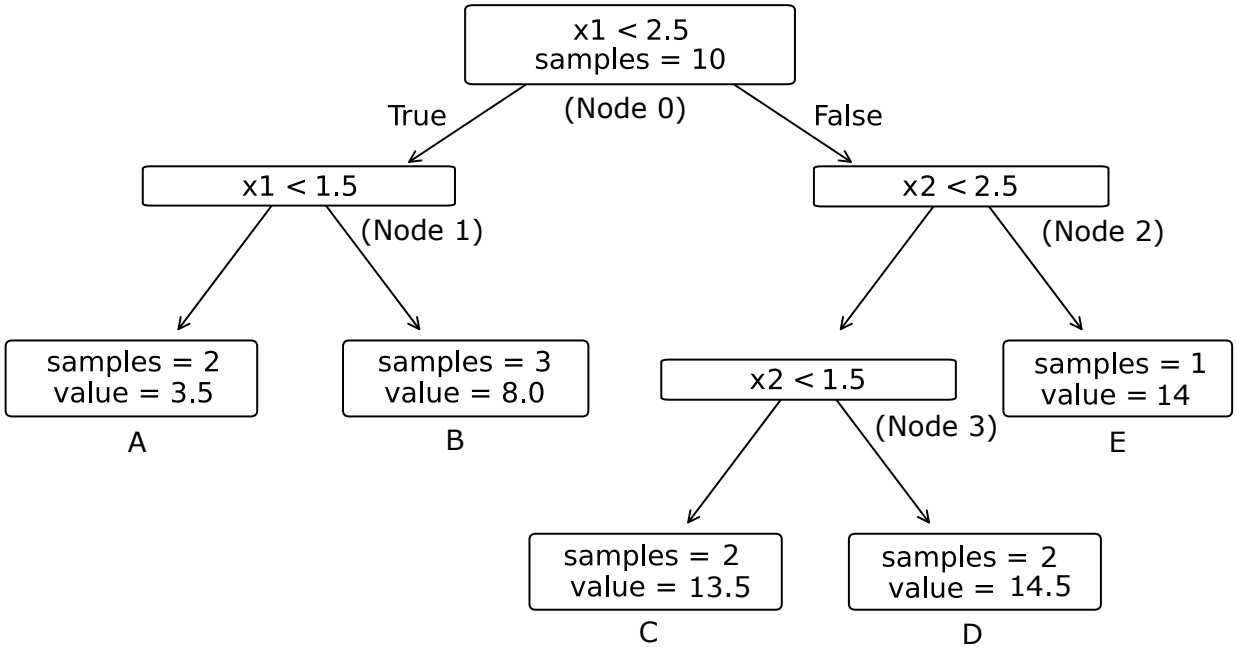
Cost Complexity Pruning Example

Here is a simple dataset with 10 observations:

x_1	x_2	y
1	1	3
1	2	4
2	1	6
2	2	8
2	3	10
3	1	12
3	2	13
3	3	14
4	1	15
4	2	16

Cost Complexity Pruning Example

Assume we build the following tree from this dataset.



Cost Complexity Pruning Example

Here are all the leaves (terminal nodes):

- Leaf A: (1,1)→ 3 and (1,2)→4, mean=3.5, error = 0.5
- Leaf B: (2,1)→ 6, (2,2)→ 8, and (2,3)→10, mean=8, error = 8
- Leaf C: (3,1)→ 12, and (4,1)→15, mean=13.5, error = 4.5
- Leaf D: (3,2)→ 13, and (4,2)→16, mean=14.5, error = 4.5
- Leaf E: (3,3)→ 14, mean=14, error = 0

Cost Complexity Pruning Example

- For cost-complexity pruning, we can compute the cost-complexity value α for each internal node t.

$$\alpha_t = \frac{R(t) - R(T_t)}{|T_t| - 1}$$

- $R(T_t)$: total error of the subtree rooted at node t
- $R(t)$: error at node t if it becomes a leaf
- $|T_t|$: number of terminal nodes in the subtree T_t

Cost Complexity Pruning Example

Internal Node 3 $[x_2 < 1.5]$

- Children: Leaf C and Leaf D.
- $R(T_t) = 4.5 + 4.5 = 9$
- Pruning node 3: node would cover $y = [12, 13, 15, 16]$, mean = 14
- $R(t) = (12 - 14)^2 + (13 - 14)^2 + (15 - 14)^2 + (16 - 14)^2 = 10$
- From that, we obtain,

$$\alpha_3 = \frac{R(t) - R(T_t)}{|T_t| - 1} = \frac{10 - 9}{2 - 1} = 1$$

Cost Complexity Pruning Example

Internal Node 2 $[x_2 < 2.5]$

- Children: Leaf C, Leaf D, and Leaf E.
- $R(T_t) = 4.5 + 4.5 + 0 = 9$
- Pruning node 2: node would cover $y = [12, 13, 14, 15, 16]$, mean = 14
- $R(t) = (12-14)^2 + (13-14)^2 + (14-14)^2 + (15-14)^2 + (16-14)^2 = 10$
- From that, we obtain,

$$\alpha_2 = \frac{R(t) - R(T_t)}{|T_t| - 1} = \frac{10-9}{3-1} = 0.5$$

Cost Complexity Pruning Example

Internal Node 1 $[x_1 < 1.5]$

- Children: Leaf A and Leaf B.
- $R(T_t) = 0.5 + 8 = 8.5$
- Pruning node 1: node would cover $y = [3, 4, 6, 8, 10]$, mean = 6.2
- $R(t) = (3-6.2)^2 + (4-6.2)^2 + (6-6.2)^2 + (8-6.2)^2 + (10-6.2)^2 = 32.8$
- From that, we obtain,

$$\alpha_1 = \frac{R(t) - R(T_t)}{|T_t| - 1} = \frac{32.8-8.5}{2-1} = 24.3$$

Cost Complexity Pruning Example

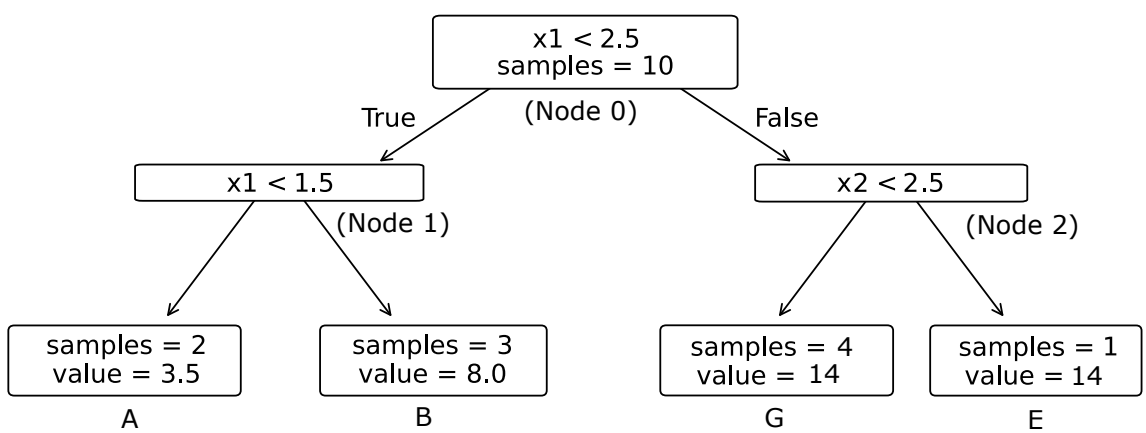
Root Node 0 $[x_1 < 2.5]$

- Children: Node 1 and Node 2.
- $R(T_t) = 8.5 + 9 = 17.5$
- Pruning node 0: node would cover all y values, mean = 10.1
- $R(t) = 190.49$
- From that, we obtain,

$$\alpha_0 = \frac{R(t) - R(T_t)}{|T_t| - 1} = \frac{190.49 - 17.5}{5 - 1} = 43.24$$

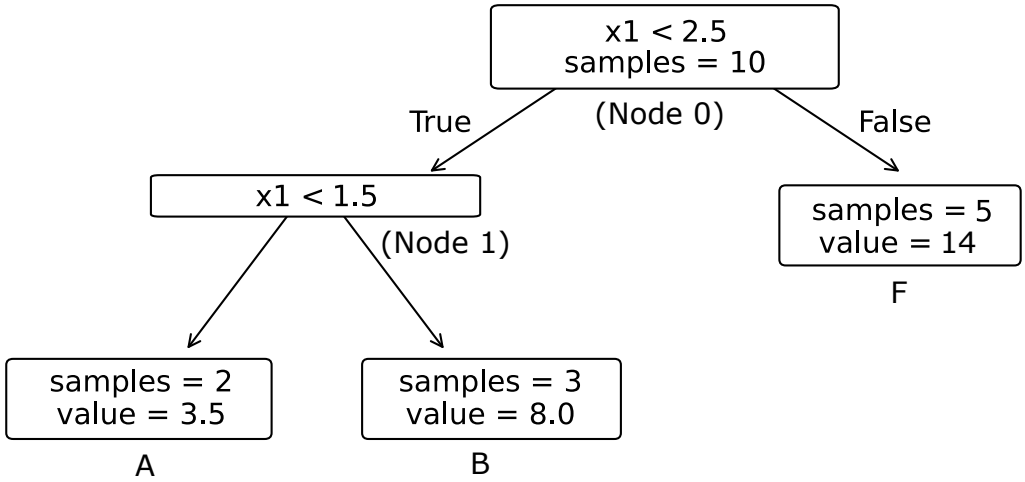
Cost Complexity Pruning Example

- Now, for a fixed $\alpha = 1$, we prune all the nodes with $\alpha \leq 1$. We can prune node 3 only or prune both nodes 2 and 3 since $\alpha_2 = 0.5$ and $\alpha_3 = 1$
- If we prune node 3 only, we will have a subtree with 3 internal nodes and 4 terminal nodes.

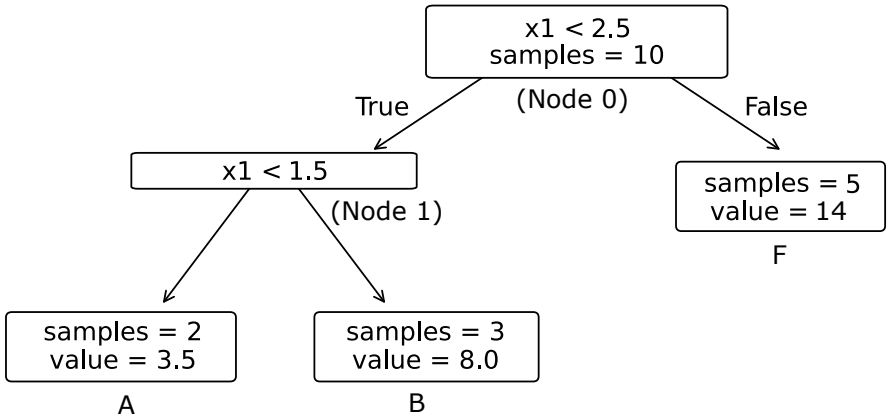


Cost Complexity Pruning Example

- If we prune nodes 2 and 3, we will have a subtree with 2 internal nodes and 3 terminal nodes.



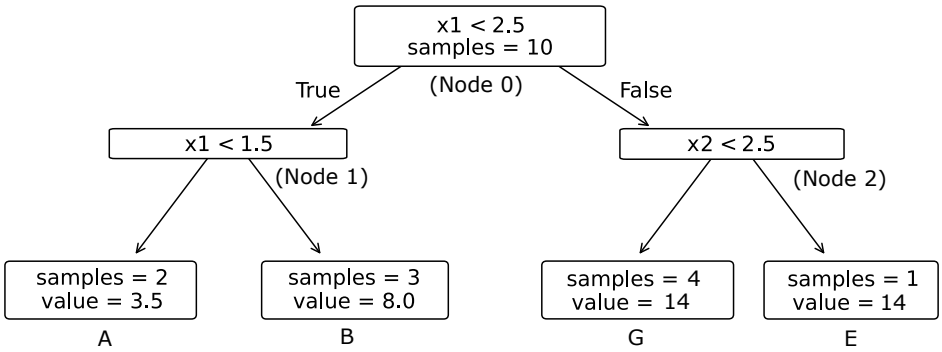
Cost Complexity Pruning Example



- As an example, we will compute $C_\alpha(T)$ for the subtree when nodes 2 and 3 are both pruned:

$$C_\alpha(T) = \sum_{m=1}^{|T|} \sum_{x_i \in R_m} \left(y_i - \hat{y}_{R_m}\right)^2 + \alpha|T| = 0.5 + 8 + 10 + 3 = 21.5$$

Cost Complexity Pruning Example



- The method to compute $C_\alpha(T)$ for the subtree when only node 3 is pruned is the same.

$$C_\alpha(T) = \sum_{m=1}^{|T|} \sum_{x_i \in R_m} \left(y_i - \hat{y}_{R_m}\right)^2 + \alpha|T| = 0.5 + 8 + 10 + 0 + 4 = 22.5$$

Cost Complexity Pruning Example

- As shown, if we prune nodes 2 and 3, we obtain a lower $C_\alpha(T)$. Thus, this is the subtree we choose for $\alpha = 1$.
- For a different α, we will obtain a different set of subtrees, and we can choose the best from them.
- At the end, we obtain a sequence of best subtrees for all the values of α.

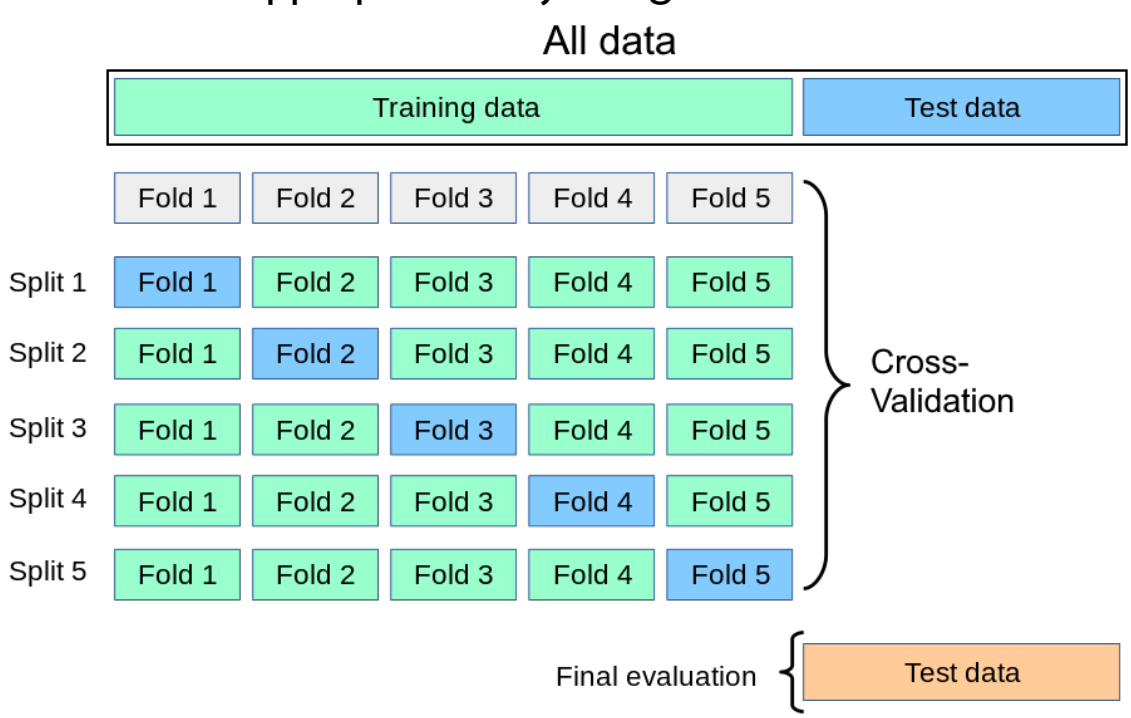
Tree-Based Methods for Regression

Tree Pruning Algorithm

1. Use recursive binary splitting to grow a large tree on the training data, stopping only when each terminal node has fewer than some minimum number of observations
2. Apply cost complexity pruning to the large tree in order to obtain a sequence of best subtrees, as a function of α.
3. Use K-fold cross-validation to choose α
4. Return the subtree from Step 2 that corresponds to the chosen α

Tree-Based Methods for Regression

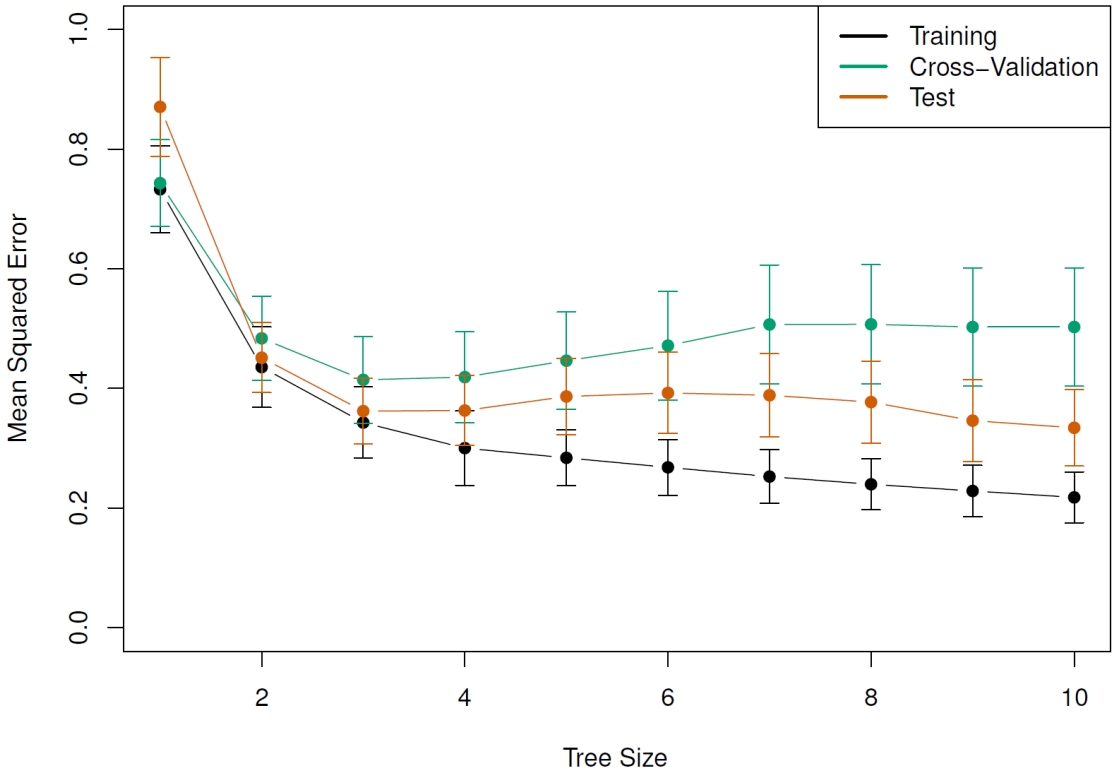
- We can choose an appropriate α by using K-fold cross-validation



Tree-Based Methods for Regression: K-fold Cross-Validation

- Assume we have 5 folds ($k = 5$) and three potential α values: $\alpha_1, \alpha_2, \alpha_3$.
- Split the training dataset into 5 parts (folds): A, B, C, D, E. For each α_i,
  1. Train on A+B+C+D, validate on E.
  2. Train on A+B+C+E, validate on D.
  3. Train on A+B+D+E, validate on C.
  4. Train on A+C+D+E, validate on B.
  5. Train on B+C+D+E, validate on A.
  6. Compute the average score (e.g., accuracy) for α_i
- Compare the average scores of all three α_i to determine the best
- A full cross-validation (where $k = 2, \ldots, K$) offers a more robust estimate of the model's performance.

Tree-Based Methods for Regression: Tree Pruning



Tree-Based Methods for Classification

- The strategy for building and pruning classification trees is very similar to the one for regression trees
- A classification tree is used to predict a qualitative response rather than a quantitative one
- For a regression tree, the predicted response for an observation is given by the mean response of the observations that belong to the same terminal node.
- In contrast, for a classification tree, we predict that each observation belongs to the most commonly occurring class of observations in the region to which it belongs.

Tree-Based Methods for Classification

- Besides the class, we are also interested in the class proportions among the training observations that fall into that region
- For classification, RSS cannot be used as a criterion for making the binary splits.
- A natural alternative to RSS is the classification error rate.

Tree-Based Methods for Classification

- The classification error rate is the fraction of the observations in that region that do not belong to the most common class:

$$E = 1 - \max_k(\hat{p}_{mk})$$

  where $\hat{p}_{mk}$ represents the proportion of training observations in the m-th region that are from the k-th class.
- However, classification error is not sufficiently sensitive for tree-growing, and in practice we prefer two other measures

Tree-Based Methods for Classification

- The Gini index (a measure of total variance across the K classes)

$$G = \sum_{k=1}^{K} \hat{p}_{mk}(1 - \hat{p}_{mk})$$

- An alternative to the Gini index is entropy,

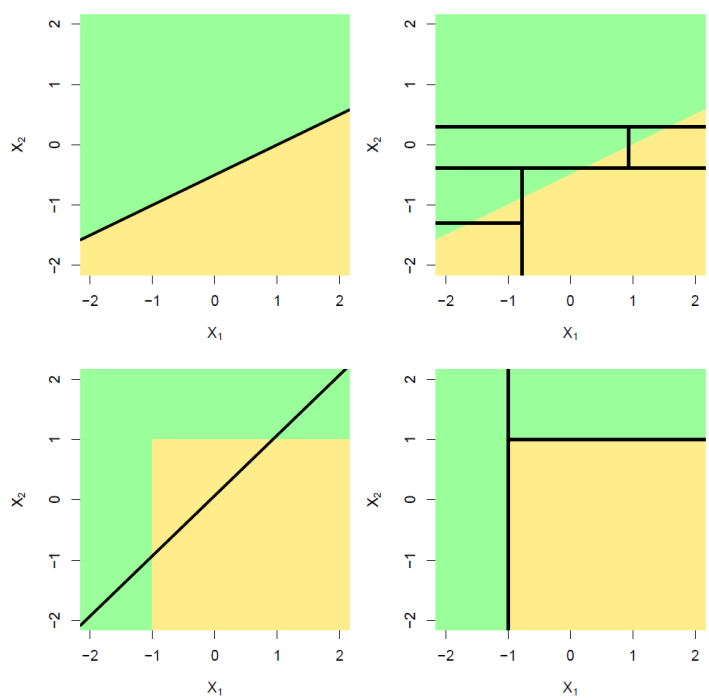
$$D = -\sum_{k=1}^{K} \hat{p}_{mk} \log \hat{p}_{mk}$$

Tree-Based Methods for Classification

- When building a classification tree, either the Gini index or the entropy are typically used to evaluate the quality of a particular split
- Any of these three measures might be used when pruning the tree, but the classification error rate is preferable if prediction accuracy of the final pruned tree is the goal
- In all the examples, we have assumed that the feature variables are continuous. However, decision trees can be constructed even with qualitative predictor variables (blood type)

Tree-Based Methods for Classification

- Tree-based models versus linear models



Tree-Based Methods for Classification

Advantages of trees

- Trees are the most easily interpretable method
- Arguably, trees more closely mirror human decision-making
- Trees are easy to display graphically
- Trees can handle qualitative features and ordered qualitative features without needing to create dummy variables

Tree-Based Methods for Classification

Disadvantages of trees

- Unfortunately, trees generally do not have the same level of predictive accuracy as some of the other regression and classification approaches
- Trees tend to have high variance. A small change in the data can cause a large change in the final estimated tree

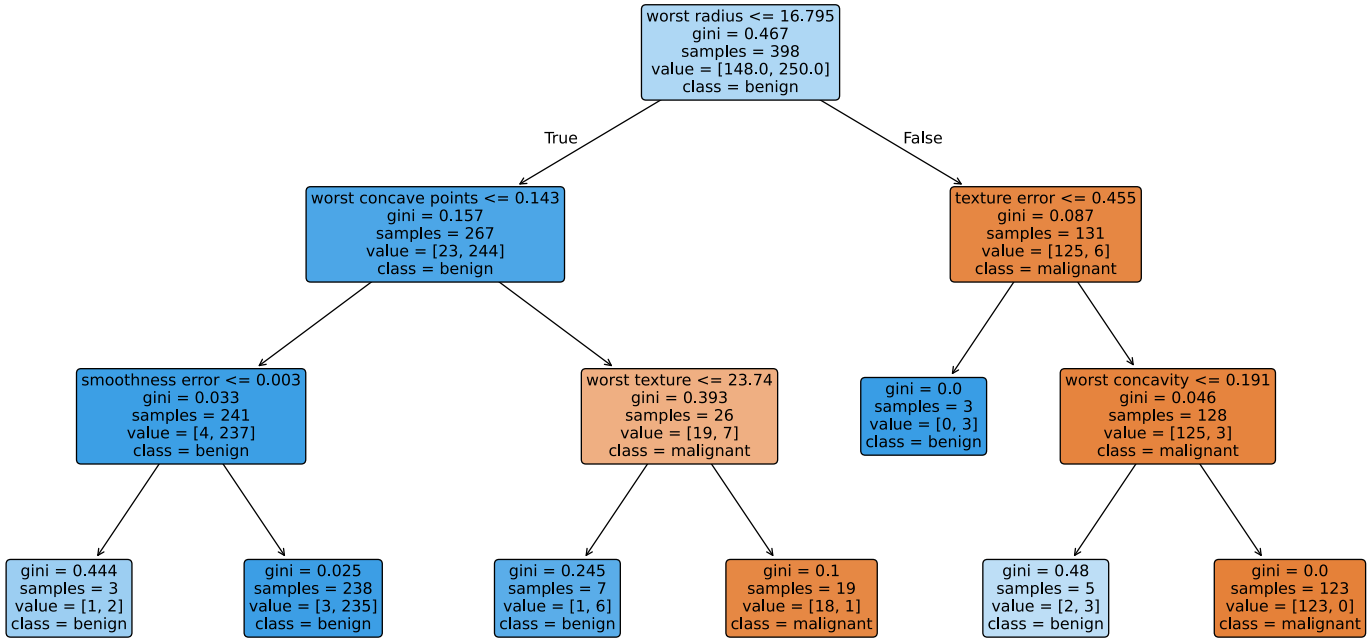
Tree-Based Methods for Classification

Python Implementation with Scikit-Learn

```python
# Load dataset
data = load_breast_cancer()
X, y = data.data, data.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)

# Train a single decision tree
tree = DecisionTreeClassifier(max_depth=3, random_state=42)
tree.fit(X_train, y_train)
acc_tree = accuracy_score(y_test, tree.predict(X_test))

print(f"Test Accuracy of the Decision Tree: {acc_tree:.3f}")
```


Tree-Based Methods for Classification

Decision Tree (Depth = 3) for Breast Cancer Dataset



Bagging

- Recall that tree-based methods also suffer from high variance.
- If we split the training dataset into two parts randomly, and fit a decision tree to both halves, the results could be quite different.
- In contrast, a method with low variance will yield similar results if applied repeatedly to distinct data sets, e.g., linear regression
- Bootstrap aggregation, or bagging, is a general-purpose procedure for reducing the variance of a statistical learning method
- It is particularly useful and frequently used for decision trees

Bagging

- Given a set of n independent observations $x_1, \ldots, x_n$, each with variance σ^2, the variance of the mean $\overline{x}$ of the observations is σ^2/n
- As an example, consider a set of 10 independent observations:

$$[8, 10, 12, 9, 11, 13, 7, 10, 12, 9]$$

- Mean of the observations: $\overline{x} = 10.1$
- The variance of individual observations: $\sigma^2 = \sum(x_i - \overline{x})^2/10 = 3.29$
- Then, the variance of the mean: $\mathrm{Var}(\overline{x}) = 3.29/10 = 0.329$
- Thus, averaging a set of observations reduces variance

Bagging

- This provides us a method to reduce the variance and increase the test accuracy of a learning method
- Consider N training data sets, we can build a separate prediction model $h(x)$ using each training set,

$$\hat{h}^1(x),\ \hat{h}^2(x),\ \cdots,\ \hat{h}^N(x)$$

- Then, we average them in order to obtain a low-variance model

$$\hat{h}_{\text{avg}}(x) = \frac{1}{N}\sum_{i=1}^{N}\hat{h}^i(x)$$

Bagging

- Of course, this is not practical because we generally do not have access to multiple training sets.
- Instead, we can bootstrap, by taking repeated samples from the (single) training dataset.
- With that, we generate N different bootstrapped training datasets.
- We then train our model on each bootstrapped data set to get $\hat{h}^i(x)$, and finally average all the predictions to obtain a low-variance model

$$\hat{h}_{\text{bag}}(x) = \frac{1}{N}\sum_{i=1}^{N}\hat{h}^i(x)$$

Bagging

- While bagging can improve predictions for many regression methods, it is particularly useful for decision trees.
- To apply bagging to regression trees, we construct N regression trees using N bootstrapped data sets, and average the resulting predictions.
- These trees are grown deep, and are not pruned. Hence each individual tree has high variance, but low bias.
- Averaging these N trees reduces the variance.
- Bagging has been demonstrated to give impressive improvements in accuracy by combining together hundreds or even thousands of trees.

Bagging

Python Implementation with Scikit-Learn

```
# Train high-variance single decision tree
tree = DecisionTreeClassifier(max_depth=3, random_state=42)
tree.fit(X_train, y_train)
acc_tree = accuracy_score(y_test, tree.predict(X_test))

# Train bagging classifier with full trees
bagging = BaggingClassifier(
    estimator=DecisionTreeClassifier(),  # full trees
    n_estimators=100,
    bootstrap=True,
    random_state=42
)
bagging.fit(X_train, y_train)
acc_bagging = accuracy_score(y_test, bagging.predict(X_test))
```

Bagging

Test accuracy with and without bagging:

```
Test Accuracy of the Decision Tree: 0.924
Test Accuracy of Bagging (100 full trees): 0.947
```